

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3074 – Minería de Datos

Sección 20



Proyecto Avances

Modelos con un algoritmo

Diego López – 23242

Diego Rosales – 23258

Erick Guerra – 23208

Hugo Barillas – 23306

José López – 23773

Milton Polanco – 23471

GUATEMALA, 15 de mayo del 2026

Índice

1 Fase 1 – Investigación Teórica	2
1.1 Definición Formal del Algoritmo	2
1.1.1 SVM Tradicional (supervisado)	2
1.1.2 Transductive SVM (TSVM)	2
1.1.3 Laplacian SVM (LapSVM)	3
1.2 Ventajas y Limitaciones	4
1.2.1 SVM Tradicional	4
1.2.2 Transductive SVM	4
1.2.3 Laplacian SVM	5
1.3 Tipos de Variables de Entrada y Criterios Estadísticos Recomendados	5
1.3.1 Tipos de variables de entrada	5
1.3.2 Normalización	6
1.3.3 Técnicas de procesamiento adicionales	6
2 Fase 2 – Selección y Análisis del Dataset	7
2.1 Descripción del Dataset	7
2.2 Exploración de Datos	7
2.3 Preprocesamiento	9
3 Fase 3 – Diseño Experimental	10
3.1 División de los Datos	10
3.2 Escenarios de Etiquetado	10
3.3 Métricas de Evaluación	11
4 Fase 4 – Experimentación	11
4.1 Resultados por Escenario de Etiquetado	11
4.2 Análisis de Sensibilidad de Hiperparámetros	12
4.2.1 TSVM	12
4.2.2 LapSVM	13
5 Fase 5 – Visualización y Discusión de Resultados	14
5.1 Curvas de Desempeño	14
5.2 Matrices de Confusión	15
5.3 Predicción vs Valor Real	17
5.4 Discusión Crítica	17
5.5 Conclusiones	18

1. Fase 1 – Investigación Teórica

1.1. Definición Formal del Algoritmo

1.1.1. SVM Tradicional (supervisado)

El *Support Vector Machine* (SVM) es un algoritmo de aprendizaje supervisado cuyo objetivo es encontrar un hiperplano que separe dos clases de datos con el mayor margen posible [1], es decir, la distancia entre el hiperplano y los puntos más cercanos de cada clase debe ser lo más grande que se pueda, los puntos que quedan justo en el borde de ese margen se llaman vectores de soporte y son los que realmente definen dónde queda la frontera de decisión.

Supuestos principales del SVM tradicional:

- Todos los datos de entrenamiento tienen etiqueta conocida ($y_i \in \{-1, +1\}$),
- Existe al menos un hiperplano que puede separar las clases (o se usa una versión flexible con parámetro de penalización C cuando no son linealmente separables),
- Los datos siguen una distribución que permite generalizar bien a datos nuevos.

Función objetivo (forma primal):

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

sujeito a $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$ y $\xi_i \geq 0$, donde $\mathbf{w}$ es el vector normal al hiperplano, b es el sesgo, ξ_i son variables de holgura que permiten clasificaciones incorrectas y C controla cuánto se penaliza cada error.

Tipo de aprendizaje: inductivo, el modelo aprende una regla general a partir de los datos etiquetados y luego aplica esa regla a cualquier dato nuevo.

1.1.2. Transductive SVM (TSVM)

La *Transductive SVM* es una extensión semi-supervisada del SVM [2], en la que se trabaja con un conjunto de datos etiquetados $L = \{(\mathbf{x}_i, y_i)\}$ y un conjunto de datos sin etiqueta $U = \{\mathbf{x}_j\}$, el algoritmo no intenta aprender una regla general para cualquier dato futuro, sino que toma decisiones directamente sobre los puntos no etiquetados que ya están disponibles.

Función objetivo (forma extendida):

$$\min_{\mathbf{w}, b, \hat{y}_j, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C_L \sum_{i \in L} \xi_i + C_U \sum_{j \in U} \xi_j^*$$

sujeto a las restricciones de clasificación correcta tanto en L como en U (con etiquetas $\hat{y}_j$ asignadas por el propio modelo), donde C_L y C_U son parámetros que controlan la importancia de los errores en datos etiquetados y no etiquetados respectivamente.

Supuestos principales:

- Los datos no etiquetados pertenecen a la misma distribución que los etiquetados,
- El hiperplano óptimo debe pasar por zonas de baja densidad de puntos (supuesto de baja densidad o *cluster assumption*),
- Los datos no etiquetados ayudan a definir mejor la frontera de decisión cuando los datos etiquetados son pocos.

Tipo de aprendizaje: transductivo, el modelo no generaliza a datos futuros desconocidos, sino que predice directamente las etiquetas de los puntos no etiquetados que ya están en el conjunto U .

1.1.3. Laplacian SVM (LapSVM)

La *Laplacian SVM* combina la idea del margen máximo de SVM con la estructura geométrica de los datos, representada mediante un grafo [3], en este grafo cada punto (etiquetado o no) es un nodo y las conexiones entre nodos reflejan qué tan parecidos son dos puntos entre sí, la información de ese grafo se resume en la matriz Laplaciana L , que captura cómo se distribuyen los datos en el espacio.

Función objetivo:

$$\min_{f \in \mathcal{H}} \frac{1}{n_L} \sum_{i=1}^{n_L} V(y_i, f(\mathbf{x}_i)) + \gamma_A \|f\|_{\mathcal{H}}^2 + \gamma f^\top L f$$

donde V es la función de pérdida (por ejemplo, pérdida bisagra), γ_A controla la complejidad del modelo en el espacio de características, γ controla cuánto se penaliza que la función f no sea suave sobre el grafo, y $f^\top L f$ mide qué tan irregular es la función en la estructura del grafo.

Supuestos principales:

- Puntos cercanos en el espacio de entrada deben tener etiquetas similares (*smoothness assumption*),

- La geometría del espacio (captada por el grafo) es útil para definir la frontera de decisión,
- La construcción del grafo es correcta y representa bien la estructura real de los datos.

Tipo de aprendizaje: semi-supervisado inductivo, a diferencia de la TSVM, el LapSVM aprende una función que sí puede aplicarse a datos nuevos.

1.2. Ventajas y Limitaciones

1.2.1. SVM Tradicional

Ventajas:

- Funciona muy bien cuando los datos son linealmente separables o cuando se usa un kernel adecuado,
- Es robusto frente a datos con ruido gracias al margen máximo,
- Existe mucha documentación, herramientas y librerías disponibles.

Limitaciones:

- Necesita todos los datos etiquetados, lo que puede ser costoso o difícil de conseguir,
- No aprovecha la información de datos sin etiqueta,
- Puede no funcionar bien cuando los datos etiquetados son muy pocos.

1.2.2. Transductive SVM

Ventajas:

- Usa los datos sin etiqueta para mejorar la frontera de decisión,
- Es útil cuando se tienen pocos datos etiquetados pero muchos sin etiqueta,
- Puede mejorar considerablemente el rendimiento respecto al SVM tradicional en esos casos.

Limitaciones:

- El problema de optimización es no convexo, lo que hace difícil encontrar la mejor solución y el proceso puede ser lento,

- Los resultados dependen mucho de los parámetros C_L y C_U ,
- No generaliza a datos futuros que no estaban en el conjunto U original,
- Si el supuesto de baja densidad no se cumple, el rendimiento puede empeorar.

1.2.3. Laplacian SVM

Ventajas:

- Usa la estructura geométrica de los datos para guiar el aprendizaje,
- A diferencia de la TSVM, sí puede predecir para datos nuevos,
- El problema de optimización es convexo, lo que facilita encontrar la solución óptima.

Limitaciones:

- Construir el grafo y calcular la matriz Laplaciana puede ser costoso en tiempo y memoria cuando hay muchos datos,
- La calidad del modelo depende de cómo se construya el grafo (parámetros del vecindario),
- Tiene tres parámetros a ajustar (γ_A , γ_I y el kernel), lo que complica la selección del modelo.

1.3. Tipos de Variables de Entrada y Criterios Estadísticos Recomendados

1.3.1. Tipos de variables de entrada

Los algoritmos SVM y sus variantes semi-supervisadas trabajan principalmente con variables numéricas continuas, ya que la función objetivo y las restricciones están definidas sobre vectores en un espacio real $\mathbb{R}^d$, sin embargo, variables categóricas también pueden usarse siempre que se transformen primero a una representación numérica (por ejemplo, codificación *one-hot*).

En resumen, los tipos de entrada compatibles son:

- Variables continuas (numéricas): el caso más directo y recomendado,
- Variables binarias o categóricas: después de aplicar una codificación adecuada,
- Datos de texto o imagen: después de extraer características numéricas representativas.

1.3.2. Normalización

Los SVM son sensibles a la escala de las variables porque el margen se calcula usando distancias [4], si una variable tiene valores muy grandes comparado con otras, dominará el cálculo del margen y el modelo aprenderá sesgado hacia esa variable, por eso se recomienda siempre normalizar o estandarizar los datos antes de entrenar.

Las técnicas más comunes son:

- **Estandarización (Z-score):** transforma cada variable para que tenga media cero y desviación estándar uno,

$$x' = \frac{x - \mu}{\sigma}$$

es la opción preferida cuando las variables no tienen una distribución uniforme y cuando se trabaja con kernels RBF o polinomial,

- **Normalización Min-Max:** escala los valores al rango $[0, 1]$,

$$x' = \frac{x - x_{\text{mín}}}{x_{\text{máx}} - x_{\text{mín}}}$$

es útil cuando se conoce el rango de los datos y no hay valores atípicos extremos.

1.3.3. Técnicas de procesamiento adicionales

Además de la normalización, se recomiendan las siguientes prácticas antes de aplicar estos algoritmos:

- **Manejo de valores faltantes:** los SVM no toleran entradas con valores nulos, por lo que se deben imputar (por ejemplo, con la media o mediana de la variable) o eliminar esas filas,
- **Reducción de dimensionalidad:** si el número de variables es muy grande, técnicas como PCA (*Principal Component Analysis*) pueden ayudar a reducir el ruido y el tiempo de cómputo antes de entrenar el modelo,
- **Balanceo de clases:** cuando una clase tiene muchos más ejemplos que la otra, el modelo puede quedar sesgado, técnicas como *oversampling* (SMOTE) o ajustar los pesos de clase (C por clase) ayudan a corregir este problema,
- **Selección del kernel:** para datos no linealmente separables se recomienda probar el kernel RBF (Gaussiano) como punto de partida, ya que funciona bien en muchos casos

sin necesitar mucho ajuste manual,

- **Validación cruzada:** para elegir los parámetros C , γ (en RBF), C_L y C_U se recomienda usar validación cruzada con búsqueda en cuadrícula (*grid search*).

2. Fase 2 – Selección y Análisis del Dataset

2.1. Descripción del Dataset

El dataset utilizado es el *Adult Census Income* [5], que fue tomado del repositorio UCI Machine Learning Repository y está basado en datos del censo de Estados Unidos de 1994, su objetivo es predecir si una persona gana más o menos de 50,000 dólares al año a partir de características demográficas y laborales, lo que lo convierte en un problema de clasificación binaria.

En su versión original el dataset cuenta con 48,842 filas y 15 variables, luego del proceso de limpieza (eliminación de duplicados y filas con valores nulos) quedaron 45,175 filas útiles para el análisis, el conjunto de variables se redujo a 12 después de remover *fnlwgt* (un peso censal que no aporta información predictiva) y *education* (que es redundante con *education_num*, su versión ordinal).

Las variables finales se dividen en:

- **Númericas (5):** *age*, *education_num*, *capital_gain*, *capital_loss*, *hours_per_week*,
- **Catóricas (7):** *workclass*, *marital_status*, *occupation*, *relationship*, *race*, *sex*, *native_country*,
- **Variable objetivo:** *income*, con dos valores posibles: $\leq 50K$ (clase 0) y $> 50K$ (clase 1).

2.2. Exploración de Datos

Balance de clases. El dataset presenta un desbalance entre las dos clases, ya que aproximadamente el 75 % de los registros pertenecen a la clase $\leq 50K$ y solo el 25 % restante a la clase $> 50K$, esto es importante tenerlo en cuenta a la hora de interpretar las métricas, ya que un modelo que siempre predice la clase mayoritaria ya obtendría un 75 % de accuracy sin aprender nada útil.

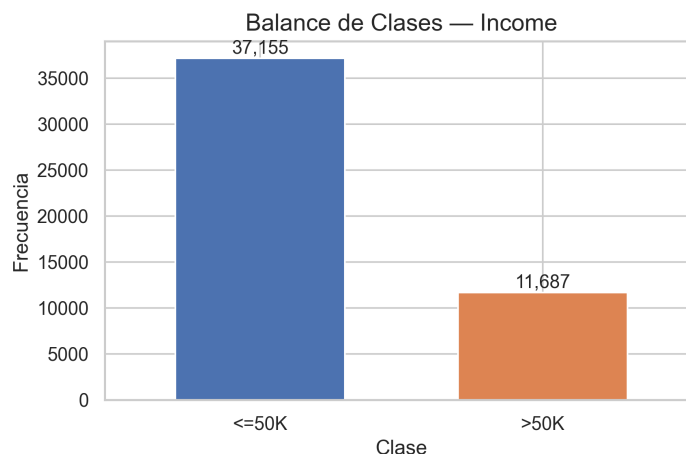


Figura 1: Balance de clases en el dataset Adult Census Income.

Distribuciones de variables numéricas. La variable *age* tiene una distribución aproximadamente normal con valores entre 17 y 90 años, *education_num* se concentra mayoritariamente entre los valores 9 y 13 (que corresponden a niveles de bachillerato y universidad), y *hours_per_week* tiene un pico claro alrededor de las 40 horas, por otro lado, *capital_gain* y *capital_loss* están fuertemente sesgadas hacia la derecha, ya que la gran mayoría de registros tiene valor cero en ambas variables y solo una pequeña fracción reporta ganancias o pérdidas de capital significativas.

Correlación entre variables numéricas. El mapa de calor muestra que las correlaciones entre variables numéricas son en general bajas, la correlación más destacada es entre *education_num* y el nivel de ingresos, lo que indica que el nivel educativo es uno de los factores más relacionados con ganar más de 50K, las variables de capital (*capital_gain* y *capital_loss*) tienen poca correlación entre sí y con las demás variables.

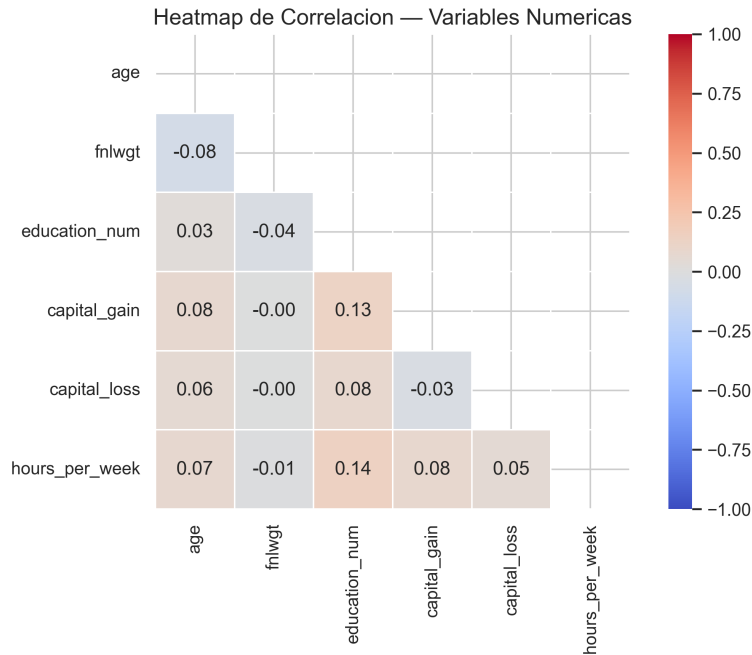


Figura 2: Mapa de calor de correlación entre variables numéricas.

Separabilidad por clase. Los boxplots muestran que las variables numéricas tienen cierta capacidad de separar las dos clases, en particular *age* y *education_num* presentan medianas claramente distintas entre la clase $\leq 50K$ y la clase $> 50K$, lo que confirma que estas variables son informativas para la clasificación.

Variables categóricas. Las variables categóricas muestran distribuciones desiguales, por ejemplo en *sex* la mayoría de registros corresponden a hombres, en *workclass* la mayor parte trabaja en el sector privado, y en *native_country* la gran mayoría son de Estados Unidos, estas variables fueron codificadas numéricamente con *LabelEncoder* para poder usarlas en los modelos SVM.

2.3. Preprocesamiento

El proceso de limpieza y transformación siguió los siguientes pasos:

1. **Eliminación de duplicados y nulos:** se removieron las filas duplicadas y aquellas con valores faltantes, pasando de 48,842 a 45,175 filas,
2. **Eliminación de variables redundantes:** se descartaron *fnlwgt* y *education*, quedando 12 variables predictoras,

3. **Codificación del target:** la variable *income* se transformó en binaria ($\leq 50K \rightarrow 0$, $> 50K \rightarrow 1$),
4. **Codificación de categóricas:** las 7 variables categóricas se transformaron a valores numéricos usando *LabelEncoder*,
5. **División train/test:** se realizó un split estratificado 80%/20%, obteniendo 36,140 filas para entrenamiento y 9,035 para prueba, manteniendo la proporción de clases en ambos conjuntos,
6. **Estandarización:** se aplicó *StandardScaler* sobre los datos de entrenamiento y se usó el mismo escalador para transformar el conjunto de prueba, garantizando que ningún modelo vea información del test durante el entrenamiento.

3. Fase 3 – Diseño Experimental

3.1. División de los Datos

Los datos procesados se dividieron en un conjunto de entrenamiento (36,140 filas) y un conjunto de prueba (9,035 filas), usando un split estratificado con semilla fija (*random_state* = 42) para garantizar que los resultados sean reproducibles y que el balance de clases se mantenga tanto en entrenamiento como en prueba.

Dentro del conjunto de entrenamiento se simuló el escenario semi-supervisado: a partir de las etiquetas reales, se seleccionó un porcentaje pequeño de filas como datos etiquetados usando muestreo estratificado, y el resto se marcó como no etiquetado (etiqueta -1), de esta manera se simula una situación realista donde obtener etiquetas es costoso y solo se tiene acceso a una fracción pequeña de ellas.

3.2. Escenarios de Etiquetado

Se definieron tres escenarios de etiquetado que varían el porcentaje de datos etiquetados disponibles para entrenamiento:

Cuadro 1: Distribución de datos etiquetados y no etiquetados por escenario.

Escenario	Etiquetados	No etiquetados	Total train
5 %	1,807	34,333	36,140
10 %	3,614	32,526	36,140
20 %	7,228	28,912	36,140

En cada escenario se entrenaron los tres modelos definidos:

- **Baseline SVC:** SVM supervisado que usa únicamente los datos etiquetados, ignorando completamente los no etiquetados,
- **TSVM:** usa los datos etiquetados y no etiquetados de forma iterativa para refinar la frontera de decisión,
- **LapSVM:** construye un grafo sobre todos los datos y usa el Laplaciano para regularizar el aprendizaje.

3.3. Métricas de Evaluación

Para comparar los modelos se usaron cuatro métricas calculadas sobre el conjunto de prueba:

- **Accuracy:** porcentaje de predicciones correctas sobre el total de casos,
- **F1-score (weighted):** promedio del F1 de cada clase ponderado por su frecuencia, es más informativo que el accuracy en presencia de desbalance de clases,
- **Precision:** de todos los casos que el modelo predice como positivos, cuántos realmente lo son,
- **Recall:** de todos los casos que son realmente positivos, cuántos detecta el modelo correctamente.

4. Fase 4 – Experimentación

4.1. Resultados por Escenario de Etiquetado

La siguiente tabla resume los resultados de los tres modelos en los tres escenarios de etiquetado, evaluados sobre el mismo conjunto de prueba de 9,035 filas.

Cuadro 2: Resultados de accuracy y F1-score por modelo y escenario de etiquetado.

Escenario	Modelo	Accuracy	F1	Precision	Recall
5 %	Baseline SVC	82.62 %	0.8082	0.8183	0.8262
	TSVM	81.22 %	0.7931	0.7997	0.8122
	LapSVM	82.58 %	0.8082	0.8174	0.8258
10 %	Baseline SVC	83.35 %	0.8204	0.8252	0.8335
	TSVM	82.51 %	0.8119	0.8148	0.8251
	LapSVM	83.38 %	0.8208	0.8254	0.8338
20 %	Baseline SVC	83.73 %	0.8249	0.8296	0.8373
	TSVM	83.07 %	0.8201	0.8213	0.8307
	LapSVM	83.80 %	0.8257	0.8303	0.8380

Los resultados muestran que LapSVM es el modelo más consistente, ya que iguala o supera ligeramente al Baseline SVC en los tres escenarios, mientras que TSVM queda por debajo del baseline en todos los casos, con una diferencia más notable al 5 % (1.4 puntos porcentuales de accuracy) que se va reduciendo al aumentar la cantidad de datos etiquetados.

4.2. Análisis de Sensibilidad de Hiperparámetros

Para analizar cómo responden los modelos ante cambios en sus hiperparámetros se realizó un *grid search* sobre una submuestra estratificada de 4,000 filas con el escenario del 10 % de etiquetado, se registraron accuracy y F1 tanto en entrenamiento como en prueba para poder identificar situaciones de overfitting o underfitting.

4.2.1. TSVM

El grid buscó sobre 18 combinaciones: $C_L \in \{0, 1, 10\}$, $C_U \in \{0, 001, 0, 01, 0, 1\}$ y $kernel \in \{linear, rbf\}$.

Los hallazgos más importantes son:

- El kernel lineal produce resultados más estables que el RBF, con una diferencia de accuracy entre entrenamiento y prueba mucho más pequeña,

- Con $C_L = 10$ y kernel RBF, el modelo muestra un overfitting claro: alcanza hasta 93.5 % de accuracy en entrenamiento pero solo 79.9 % en prueba, una brecha de casi 14 puntos porcentuales,
- Con kernel lineal y cualquier valor de C_L , el modelo se mantiene alrededor del 80 %–81 % de accuracy en prueba sin signos de overfitting,
- C_U tiene un efecto pequeño en la mayoría de configuraciones con kernel lineal, pero afecta la convergencia cuando se usa RBF.

4.2.2. LapSVM

El grid buscó sobre 27 combinaciones: $\gamma \in \{0,01, 0,1, 1,0\}$, $n_neighbors \in \{5, 7, 10\}$ y $C \in \{0,1, 1, 10\}$.

Los hallazgos más importantes son:

- $C = 0,1$ produce underfitting en casi todas las combinaciones, con accuracy en prueba cercano al 75 %, lo que indica que la penalización es demasiado baja para aprender la frontera de decisión correctamente,
- $C = 1,0$ es la configuración más equilibrada, con accuracy en prueba entre 80 % y 81.3 % y sin grandes brechas frente al entrenamiento,
- $C = 10$ genera overfitting moderado a severo, con accuracy en entrenamiento entre 93 % y 94 % pero valores en prueba de 77 %–81 %,
- γ y $n_neighbors$ tienen un efecto secundario: valores altos de γ (1.0) con $C = 10$ agravan el overfitting, mientras que con $C = 1,0$ el impacto es pequeño.

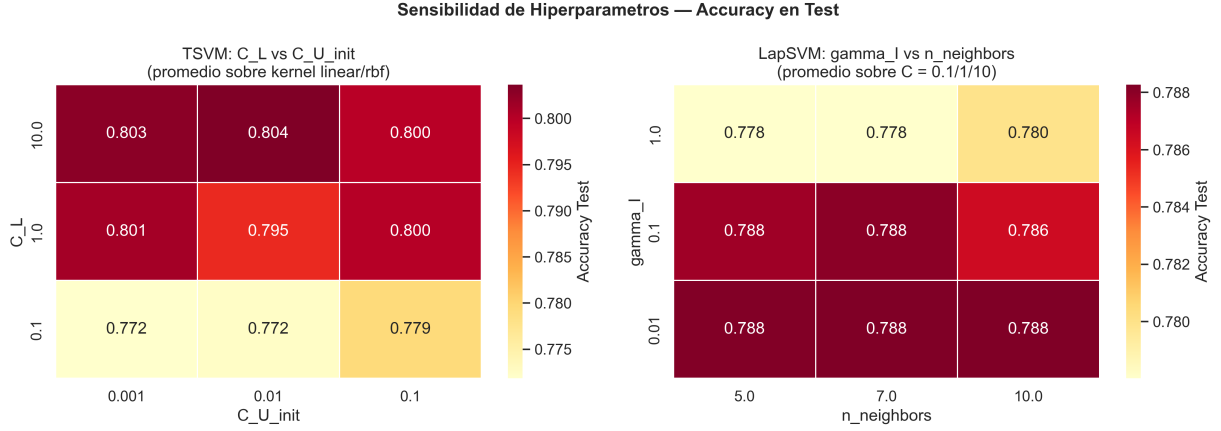


Figura 3: Heatmaps de accuracy en prueba para TSM (C_L vs C_U) y LapSVM (γ_I vs $n_neighbors$).

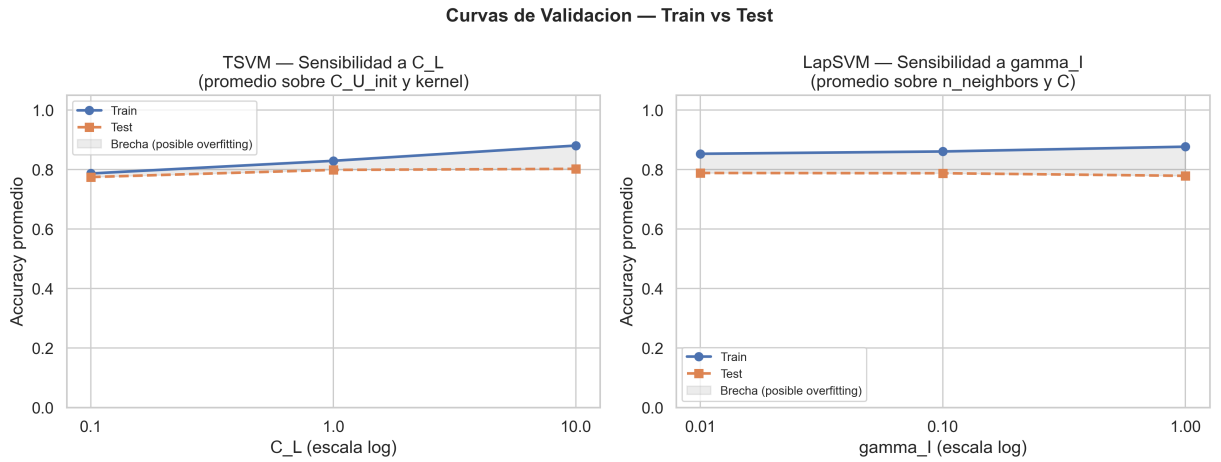


Figura 4: Curvas de validación: accuracy en entrenamiento vs prueba para distintos valores de C_L (TSM) y γ_I (LapSVM).

5. Fase 5 – Visualización y Discusión de Resultados

5.1. Curvas de Desempeño

Las curvas de desempeño muestran cómo varía el accuracy y el F1-score de cada modelo a medida que aumenta el porcentaje de datos etiquetados disponibles, lo que permite ver de un vistazo qué tan bien aprovecha cada algoritmo la información adicional.

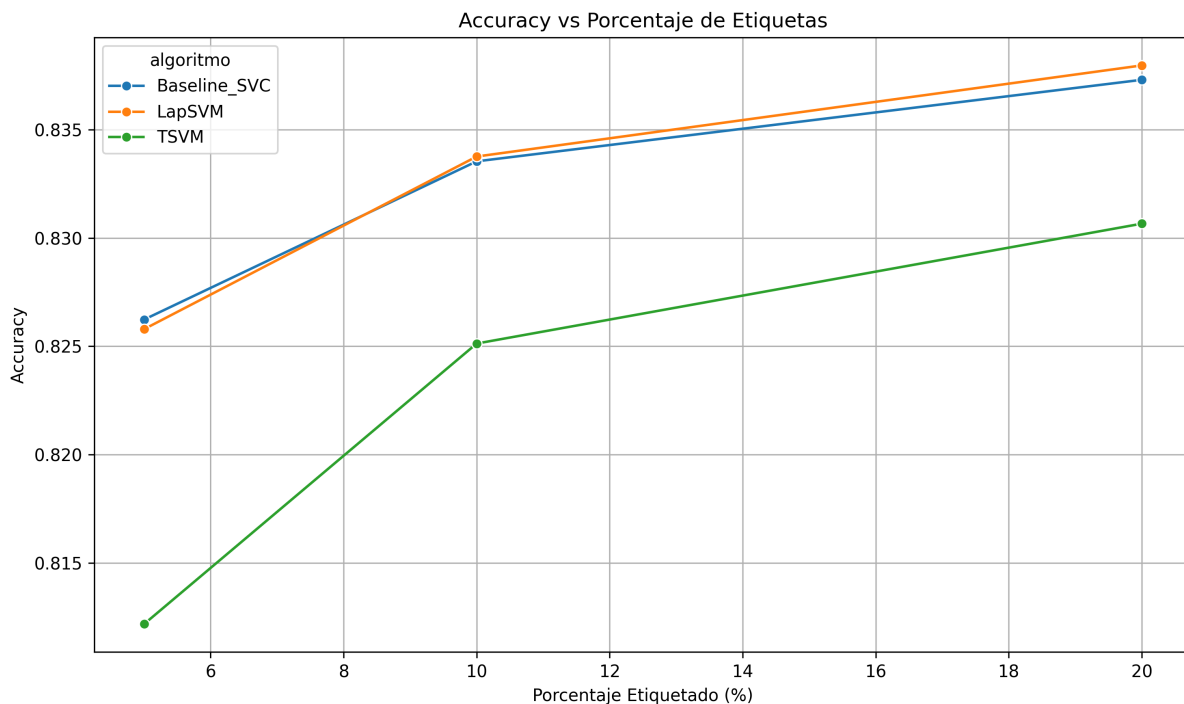


Figura 5: Curvas de accuracy de los tres modelos en función del porcentaje de etiquetado.

En la curva se observa que los tres modelos mejoran a medida que aumenta el porcentaje de datos etiquetados, lo cual es el comportamiento esperado, y que LapSVM y Baseline SVC se mantienen muy cerca entre sí en todos los escenarios mientras que TSVM se queda consistentemente por debajo.

5.2. Matrices de Confusión

Las matrices de confusión permiten ver, en detalle, cuántos casos de cada clase se predicen bien o mal, la figura muestra las 9 matrices correspondientes a los 3 modelos en los 3 escenarios de etiquetado, organizadas en una cuadrícula de 3×3 .

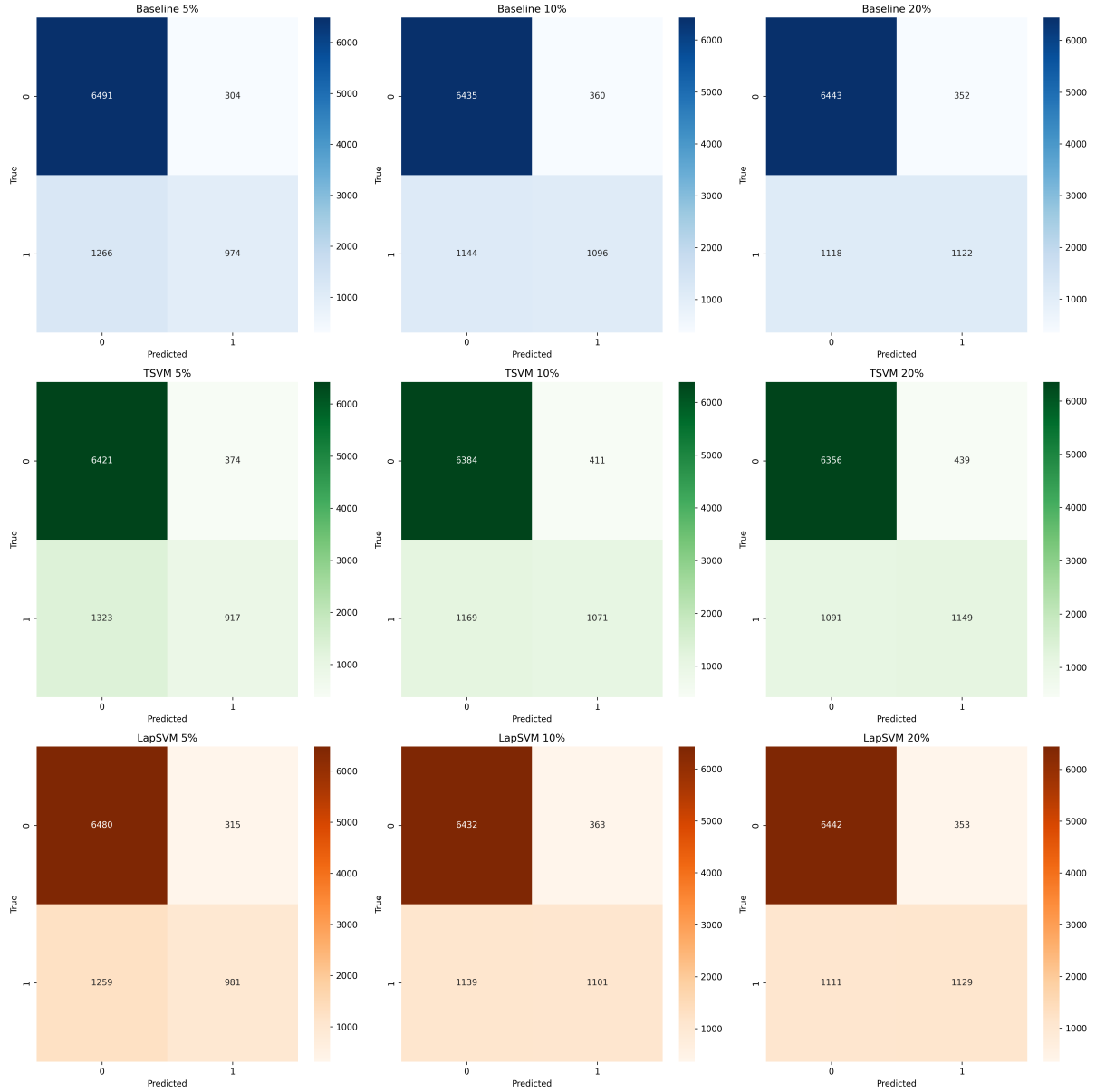


Figura 6: Matrices de confusión para los tres modelos en los tres escenarios de etiquetado (5 %, 10 %, 20 %).

En general, los tres modelos identifican bien la clase mayoritaria ($\leq 50K$), pero tienen más dificultad con la clase minoritaria ($> 50K$), lo que es esperable dado el desbalance del dataset, y a medida que aumenta la cantidad de datos etiquetados mejora la detección de la clase positiva en todos los modelos.

5.3. Predicción vs Valor Real

El siguiente gráfico compara las predicciones de los tres modelos contra los valores reales de la variable objetivo en el escenario del 10 % de etiquetado, que representa el caso intermedio, y permite visualizar dónde se concentran los errores de clasificación en cada modelo.

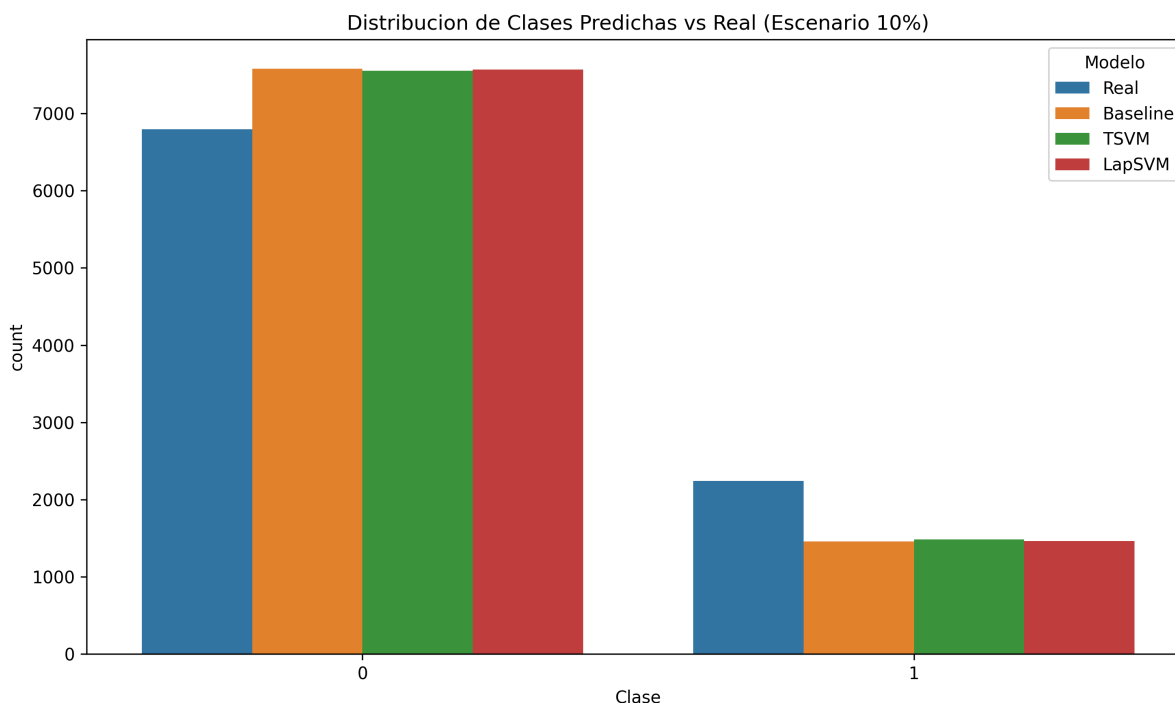


Figura 7: Predicción vs valor real para los tres modelos con 10% de datos etiquetados.

5.4. Discusión Crítica

¿Por qué LapSVM iguala al baseline? El LapSVM construye un grafo sobre todos los datos (etiquetados y no etiquetados) y usa el Laplaciano para suavizar las predicciones a lo largo de la estructura del dataset, esto hace que puntos cercanos en el espacio de características reciban predicciones similares, lo que ayuda cuando los datos etiquetados son pocos, en el caso del Adult Income, la estructura geométrica del dataset es suficientemente clara como para que este suavizado beneficie al modelo, permitiéndole alcanzar un rendimiento equivalente al del SVM supervisado incluso con solo el 5 % de los datos etiquetados.

¿Por qué TSVM queda por debajo del baseline? La TSVM es un método transductivo que asigna pseudo-etiquetas a los datos no etiquetados de forma iterativa, empezando con predicciones

del SVM inicial y refinándolas en cada vuelta, sin embargo, cuando los datos etiquetados son muy pocos y la distribución de las clases está desbalanceada (como en este dataset), las pseudo-etiquetas iniciales pueden ser incorrectas, y esos errores se propagan a través de las iteraciones en lugar de corregirse, esto explica que TSVM tenga su mayor desventaja en el escenario del 5 %, donde el SVM inicial está entrenado con muy poca información, y que esa diferencia se vaya cerrando al aumentar el porcentaje de etiquetado.

¿Cuándo conviene usar cada algoritmo? El Baseline SVC es la opción más simple y en este caso resulta muy competitivo, lo cual sugiere que cuando el kernel RBF logra capturar bien la estructura del problema, los métodos semi-supervisados no siempre ofrecen una ventaja clara, el LapSVM es una buena alternativa cuando se tiene muy poca información etiquetada pero muchos datos sin etiquetar que revelan la geometría del espacio, y la TSVM es más útil en problemas donde el supuesto de baja densidad (la frontera de decisión pasa por zonas con pocos puntos) se cumple con claridad, lo que no ocurre de forma fuerte en el Adult Income.

Análisis de robustez. Los tres modelos muestran un comportamiento estable al variar el porcentaje de etiquetado: sus métricas mejoran de forma gradual y sin caídas abruptas, lo que indica que no son frágiles ante pequeñas variaciones en la cantidad de datos etiquetados disponibles, en el análisis de sensibilidad, el único problema claro de estabilidad fue el overfitting de TSVM y LapSVM con valores de C muy altos, que se puede controlar fácilmente eligiendo $C = 1,0$ como valor predeterminado.

5.5. Conclusiones

A partir de los experimentos realizados se puede concluir que:

- LapSVM es el modelo semi-supervisado más efectivo para este problema, ya que iguala o mejora marginalmente al SVM supervisado en los tres escenarios sin necesidad de aumentar los datos etiquetados,
- TSVM no supera al baseline en ningún escenario, y su rendimiento depende fuertemente de que las pseudo-etiquetas iniciales sean confiables, lo que se dificulta con datos muy escasos y clases desbalanceadas,
- Aumentar el porcentaje de datos etiquetados del 5 % al 20 % mejora el desempeño de todos los modelos, pero la ganancia es relativamente pequeña, lo que indica que el SVM con kernel RBF ya extrae mucha información incluso con pocos datos etiquetados en este dataset,

- El análisis de sensibilidad mostró que el parámetro más crítico en ambos algoritmos semi-supervisados es C : valores demasiado bajos causan underfitting y valores demasiado altos causan overfitting, siendo $C = 1,0$ una elección equilibrada en ambos casos,
- El Adult Census Income es un dataset donde los métodos semi-supervisados no ofrecen una ventaja dramática sobre el baseline, probablemente porque la estructura del dataset ya es bastante clara con pocos ejemplos y el kernel RBF la captura bien, lo que refuerza la importancia de comparar siempre los modelos semi-supervisados contra un baseline supervisado sólido.

Referencias

- [1] GeeksforGeeks. (2025). Support Vector Machine (SVM) Algorithm. <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>
- [2] Cevikalp, H. & Franc, V. (2017). Large-scale robust transductive support vector machines. *Neurocomputing*, 235, 199–209. <https://www.sciencedirect.com/science/article/abs/pii/S0925231217300292>
- [3] Kim, J., Kim, Y. & Kim, Y. (2023). Multiclass Laplacian support vector machine with functional analysis of variance decomposition. *Computational Statistics & Data Analysis*, 187, 107804. <https://www.sciencedirect.com/science/article/abs/pii/S0167947323001251>
- [4] ScienceDirect Topics. (2022). Feature Scaling – an overview. <https://www.sciencedirect.com/topics/computer-science/feature-scaling>
- [5] Becker, B. & Kohavi, R. (1996). *Adult* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5XW20>